Neon Carbon UI — Design & Scaffold (v1)

> A lightweight, finance-grade, real-time dashboard for pricing & swaps with a dark, premium "Neon Carbon" look.

---

1) Vision & Aesthetic

Mood: carbon-black base, subtle glass, micro-glow accents, crisp typography, fast micro-motion.

- **Base colors**
- Background: `#0b0f14` (carbon)
- Panels: `#0f1620` / `#0c131b`
- Borders: `rgba(255,255,255,0.06)`
- Text: `#e6eef7` (primary), `#9fb1c9` (muted)
- **Accents**
- Cyan (primary): `#00e5ff`
- Violet (secondary): `#7c3aed`
- Feedback: Up `#22c55e`, Down `#ef4444`, Warn `#f59e0b`
- **Typography**
- UI: **Inter** (variable if available)
- Numeric/data: **JetBrains Mono**
- Enable tabular numerals: `font-feature-settings: "tnum" on, "lnum" on;`
- **Motion**
- Subtle, under 200ms, easing: `cubic-bezier(0.22, 1, 0.36, 1)`
- Use transform/opacity; avoid layout thrash
- **Background texture**
- Radial “auroras” + faint noise: cheap GPU, luxe feel

Theme tokens (CSS)

```
:root {
  --bg: #0b0f14;
  --panel: #0f1620;
  --panel-2: #0c131b;
  --border: rgba(255,255,255,0.06);
  --text: #e6eef7;
  --muted: #9fb1c9;

  --accent: #00e5ff;      /* cyan */
  --accent-2: #7c3aed;    /* violet */

  --up: #22c55e;
  --down: #ef4444;
```

```
  --warn: #f59e0b;

  --ring: rgba(0,229,255,0.35);
  --shadow: 0 10px 30px rgba(0,0,0,0.5);
}
html { color-scheme: dark; }
body {
  background:
   radial-gradient(1200px 600px at 10% -10%, rgba(124,58,237,0.15), transparent
50%),
   radial-gradient(1200px 600px at 100% 0%, rgba(0,229,255,0.08), transparent
40%),
   var(--bg);
  color: var(--text);
  font-feature-settings: "ss01" on, "tnum" on, "lnum" on;
}
.card {
  background: linear-gradient(180deg, rgba(255,255,255,0.02), rgba(255,255,255,
0.00));
  border: 1px solid var(--border);
  border-radius: 16px;
  box-shadow: var(--shadow);
  backdrop-filter: saturate(140%) blur(6px);
}
```

---

2) Stack & Libraries

- **Frontend**: React + Vite + TypeScript
- **Styling**: Tailwind CSS (+ custom tokens), **shadcn/ui** (Radix primitives), **Lucide** icons
- **Motion**: Framer Motion (micro-interactions)
- **Charts**: TradingView **Lightweight-Charts** (main), small SVG for sparklines
- **State**: Zustand (stores), TanStack Virtual (long lists)
- **Transport**: WebSocket to a thin `ui-gateway` (Rust/axum) that fans out Kafka messages

Why this stack?

- Tiny bundles, fast HMR, A11y via Radix, production-grade charting without heavy deps, simple state model.

---

3) Data Contracts (from backend → UI)

PriceSnapshot (WS `prices`)

```ts
// key: `${dex}:${pair}` e.g. "Orca:SOL/USDC"
interface PriceSnapshot {
  ts: number;        // epoch seconds
  dex: string;       // "Orca"
  pair: string;      // "SOL/USDC"
  price: number;     // last trade price
  src_sig: string;   // source tx signature
  window_secs: number; // aggregator window used
}
```

NormalizedSwap (WS `swaps`)

```ts
interface NormalizedSwap {
  ts: number;
  slot: number;
  sig: string;       // base58
  dex: string;       // "orca"
  token_in?: string; // mint
  amount_in?: number;
  token_out?: string;// mint
  amount_out?: number;
}
```

> **Note:** Gateway will read Kafka topics `prices.latest` and `orca-swaps.parsed` (or `...dex`) and broadcast JSON frames over WS.

---

4) UI Information Architecture

- **Top Bar** (sticky): brand, WS status pill, time window, quick filters
- **Left Grid** – *Live Prices*
- `PriceCard`: pair, DEX, last, Δ1m, 60s sparkline
- Click → open focus panel with full Lightweight-Chart
- **Right Column** – *Swaps Feed* (virtualized)
- row: time, pair, amount_in → amount_out, dex, link to explorer
- **Bottom (optional)** – *Arb Radar*
- carousel of top spreads across DEXes

---

5) Performance Playbook

- Batch WS updates (coalesce by key) every 250ms
- Cap visible DOM nodes (virtualize lists)
- Keep a ring buffer of 60 points per displayed sparkline
- Animate with transforms/opacity only; avoid heavy box-shadows on large elements

---

6) Frontend Scaffold (Vite + Tailwind + shadcn/ui)

Create project

```
npm create vite@latest ui-web -- --template react-ts
cd ui-web
npm i
npm i -D tailwindcss postcss autoprefixer @types/node
npx tailwindcss init -p
npm i class-variance-authority clsx tailwind-merge
npm i framer-motion zustand @tanstack/react-virtual lucide-react lightweight-
charts
```

Tailwind setup

`tailwind.config.ts`

```ts
import type { Config } from 'tailwindcss'
export default {
  darkMode: ['class'],
  content: ['./index.html','./src/**/*.{ts,tsx}'],
  theme: {
    extend: {
      colors: {
        bg: 'var(--bg)', panel: 'var(--panel)', panel2: 'var(--panel-2)',
        text: 'var(--text)', muted: 'var(--muted)',
        accent: 'var(--accent)', accent2: 'var(--accent-2)',
        up: 'var(--up)', down: 'var(--down)', warn: 'var(--warn)'
      },
      boxShadow: { neon: '0 0 0 1px var(--accent) inset, var(--shadow)' },
      borderRadius: { xl2: '1rem' }
    }
  },
  plugins: []
} satisfies Config
```

src/index.css

```
@tailwind base; @tailwind components; @tailwind utilities;
/* Neon Carbon tokens */
:root { /* (paste tokens from §1) */ }
html { color-scheme: dark; }
body { /* (paste body background from §1) */ }
.card { /* (paste .card styles from §1) */ }
```

Base layout

src/App.tsx

```
import { useEffect, useState } from 'react'
import { motion } from 'framer-motion'
import { PricesPanel } from './components/PricesPanel'
import { SwapsPanel } from './components/SwapsPanel'
import { WsStatus } from './components/WsStatus'

export default function App(){
  return (
    <div className="min-h-dvh p-4 gap-4 grid grid-cols-12 bg-[var(--bg)]">
      <header className="col-span-12 flex items-center justify-between card
p-3">
        <div className="font-semibold tracking-wide">MEVana • Neon Carbon</div>
        <WsStatus />
      </header>
      <section className="col-span-7 card p-3"><PricesPanel/></section>
      <aside className="col-span-5 card p-3"><SwapsPanel/></aside>
    </div>
  )
}
```

WS hooks & store

src/state/useStore.ts

```
import create from 'zustand'

export interface PriceSnapshot { ts:number; dex:string; pair:string;
price:number; src_sig:string; window_secs:number }
export interface NormalizedSwap { ts:number; slot:number; sig:string;
dex:string; token_in?:string; amount_in?:number; token_out?:string;
amount_out?:number }
```

```
type State = {
  prices: Record<string, PriceSnapshot>
  priceHistory: Record<string, number[]> // last 60 points per key
  swaps: NormalizedSwap[]
  upsertPrice: (p: PriceSnapshot) => void
  pushSwap: (s: NormalizedSwap) => void
}

export const useStore = create<State>((set,get)=>({
  prices: {}, priceHistory: {}, swaps: [],
  upsertPrice: (p)=> set((st)=>{
    const key = `${p.dex}:${p.pair}`
    const hist = st.priceHistory[key] ?? []
    const next = [...hist.slice(-59), p.price]
    return { prices:{...st.prices,[key]:p}, priceHistory:{...st.priceHistory,
[key]:next} }
  }),
  pushSwap: (s)=> set((st)=>({ swaps:[s, ...st.swaps].slice(0,500) }))
}))
```

`src/hooks/useWS.ts`

```
import { useEffect } from 'react'
import { useStore } from '../state/useStore'

export function useWS(){
  const upsertPrice = useStore(s=>s.upsertPrice)
  const pushSwap = useStore(s=>s.pushSwap)

  useEffect(()=>{
    const priceWS = new WebSocket('ws://localhost:8081/ws/prices')
    const swapWS  = new WebSocket('ws://localhost:8081/ws/swaps')
    priceWS.onmessage = (e)=>{ try { upsertPrice(JSON.parse(e.data)) } catch
{} }
    swapWS.onmessage  = (e)=>{ try { pushSwap(JSON.parse(e.data)) } catch {} }
    return ()=>{ priceWS.close(); swapWS.close() }
  },[upsertPrice,pushSwap])
}
```

Panels & components (sketches)

`src/components/PricesPanel.tsx`

```
import { useWS } from '../hooks/useWS'
import { useStore } from '../state/useStore'
import { PriceCard } from './PriceCard'

export function PricesPanel(){
  useWS()PriceCard
  const entries = Object.entries(useStore(s=>s.prices))
  return (
    <div className="grid grid-cols-2 gap-3">
      {entries.map(([key,p])=> <PriceCard key={key} p={p} />)}
    </div>
  )
}
```

src/components/PriceCard.tsx

```
import { useStore } from '../state/useStore'
import { motion } from 'framer-motion'

export function PriceCard({p}:{p:any}){
  const hist = useStore(s=>s.priceHistory[`${p.dex}:${p.pair}`] ?? [])
  return (
    <motion.div layout className="card p-3 hover:shadow-neon transition">
      <div className="flex items-center justify-between">
        <div className="text-sm text-muted">{p.dex}</div>
        <div className="text-xs text-muted">{new
Date(p.ts*1000).toLocaleTimeString()}</div>
      </div>
      <div className="text-lg font-semibold mt-1">{p.pair}</div>
      <div className="text-2xl font-mono tabular-nums mt-2">{p.price.toFixed(5)}
</div>
      <Sparkline points={hist} />
    </motion.div>
  )
}

function Sparkline({points}:{points:number[]}){
  if(points.length<2) return <div className="h-8" />
  const w=240,h=40; const max=Math.max(...points), min=Math.min(...points)
  const norm = (v:number)=> h - (h*(v-min))/Math.max(1e-9,(max-min))
  const step = w/(points.length-1)
  const d = points.map((v,i)=> `${i? 'L':'M'} ${i*step} ${norm(v)}`).join(' ')
  return (
    <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-10 mt-2">
      <path d={d} fill="none" stroke="var(--accent)" strokeWidth="1.5" />
    </svg>
```

```
  )
}
```

src/components/SwapsPanel.tsx

```
import { useStore } from '../state/useStore'

export function SwapsPanel(){
  const swaps = useStore(s=>s.swaps)
  return (
    <div className="space-y-2 overflow-auto max-h-[72vh] pr-1">
      {swaps.map((s)=> (
        <div key={s.sig} className="card p-2 flex items-center justify-between">
          <div className="text-sm text-muted">{new
Date(s.ts*1000).toLocaleTimeString()}</div>
          <div className="font-mono">{s.sig.slice(0,8)}…</div>
          <div className="text-sm">{s.dex.toUpperCase()}</div>
          <div className="text-sm">{s.token_in} → {s.token_out}</div>
          <div className="font-mono">{s.amount_in ?? '?'} → {s.amount_out ??
'?'}</div>
        </div>
      ))}
    </div>
  )
}
```

src/components/WsStatus.tsx

```
import { useEffect, useState } from 'react'
export function WsStatus(){
  const [ok,setOk]=useState(false)
  useEffect(()=>{
    const ws=new WebSocket('ws://localhost:8081/ws/ping')
    ws.onopen=()=>setOk(true)
    ws.onclose=()=>setOk(false)
    return ()=>ws.close()
  },[])
  return <div className={`px-2 py-1 rounded-full text-xs ${ok?'bg-green-500/20
text-green-300':'bg-red-500/20 text-red-300'}`}>{ok?'live':'offline'}</div>
}
```

---

7) Backend Gateway (Rust/axum)

Purpose: consume Kafka topics and fan out over WebSockets.

Create crate

```
cargo new ui-gateway --bin
```

`ui-gateway/Cargo.toml`

```
[package]
name = "ui-gateway"
version = "0.1.0"
edition = "2021"

[dependencies]
anyhow = "1"
axum = { version = "0.7", features=["ws"] }
axum-extra = { version = "0.9", features=["typed-header"] }
chrono = { version = "0.4", features=["serde"] }
futures = "0.3"
rdkafka = { version = "0.38", features=["cmake-build"] }
serde = { version = "1", features=["derive"] }
serde_json = "1"
tokio = { version = "1", features=["full"] }
tokio-stream = "0.1"
tracing = "0.1"
tracing-subscriber = { version = "0.3", features=["env-filter"] }
```

`ui-gateway/src/main.rs`

```
use axum::{extract::State, routing::get, Router};
use axum::extract::ws::{WebSocketUpgrade, Message as WsMessage};
use anyhow::Result;
use rdkafka::consumer::{Consumer, StreamConsumer};
use rdkafka::ClientConfig;
use std::{sync::Arc, time::Duration};
use tokio::{sync::broadcast, select};
use tokio_stream::StreamExt;

#[derive(Clone)]
struct AppState {
    prices_tx: broadcast::Sender<String>,
    swaps_tx: broadcast::Sender<String>,
```

```
}

#[tokio::main]
async fn main() -> Result<()> {
    tracing_subscriber::fmt().with_env_filter("info").init();
    let broker = std::env::var("KAFKA_BROKER").unwrap_or("localhost:
29092".into());
    let prices_topic =
std::env::var("PRICES_TOPIC").unwrap_or("prices.latest".into());
    let swaps_topic = std::env::var("SWAPS_TOPIC").unwrap_or("orca-
swaps.parsed".into());

    let (prices_tx, _) = broadcast::channel::<String>(1024);
    let (swaps_tx,  _) = broadcast::channel::<String>(1024);
    let state = AppState{ prices_tx: prices_tx.clone(), swaps_tx:
swaps_tx.clone() };

    // spawn Kafka consumers
    tokio::spawn(consumer_task(broker.clone(), prices_topic, prices_tx,
"prices"));
    tokio::spawn(consumer_task(broker.clone(), swaps_topic,  swaps_tx,
"swaps"));

    let app = Router::new()
        .route("/ws/prices", get(ws_prices))
        .route("/ws/swaps",  get(ws_swaps))
        .route("/ws/ping",   get(ws_ping))
        .with_state(state);

    let addr = "0.0.0.0:8081".parse().unwrap();
    tracing::info!("ui-gateway listening on {addr}");
    axum::Server::bind(&addr).serve(app.into_make_service()).await?;
    Ok(())
}

async fn consumer_task(broker:String, topic:String,
tx:broadcast::Sender<String>, label:&'static str){
    let c: StreamConsumer = ClientConfig::new()
        .set("bootstrap.servers", &broker)
        .set("group.id", format!("ui-{}", label))
        .set("auto.offset.reset", "latest")
        .create()
        .expect("consumer");
    c.subscribe(&[&topic]).expect("subscribe");
    tracing::info!("consuming topic={}", topic);
    loop {
        match c.recv().await {
            Ok(m) => if let Some(p) = m.payload_view::<str>().and_then(|r|
```

```
r.ok()) {
                let _ = tx.send(p.to_string());
            },
            Err(e) => tracing::warn!(?e, "kafka recv error"),
        }
    }
}

async fn ws_prices(ws: WebSocketUpgrade, State(s): State<AppState>) ->
axum::response::Response {
    ws.on_upgrade(move |mut socket| async move {
        let mut rx = s.prices_tx.subscribe();
        while let Ok(msg) = rx.recv().await { let _ =
socket.send(WsMessage::Text(msg)).await; }
    })
}
async fn ws_swaps(ws: WebSocketUpgrade, State(s): State<AppState>) ->
axum::response::Response {
    ws.on_upgrade(move |mut socket| async move {
        let mut rx = s.swaps_tx.subscribe();
        while let Ok(msg) = rx.recv().await { let _ =
socket.send(WsMessage::Text(msg)).await; }
    })
}
async fn ws_ping(ws: WebSocketUpgrade) -> axum::response::Response {
    ws.on_upgrade(move |mut socket| async move { let _ =
socket.send(WsMessage::Text("ok".into())).await; })
}
```

Run

```
KAFKA_BROKER=localhost:29092 \
PRICES_TOPIC=prices.latest \
SWAPS_TOPIC=orca-swaps.parsed \
cargo run -p ui-gateway
```

---

8) Quickstart (Full Loop)

1. Ensure Kafka topics exist: `orca-swaps`, `orca-swaps.parsed`, `prices.latest` (compacted)
2. Run **dex-stream-orca** (producer)
3. Run **normalizer-orca** (parsed output)
4. Run **price-aggregator** (publish `prices.latest`)
5. Run **ui-gateway** (WS bridge)
6. Run **ui-web** (Vite dev): `npm run dev` → open `http://localhost:5173`

You should see price cards filling in and swaps scrolling in real time.

---

9) Assets (free & permissive)

- Icons: **Lucide**, **Tabler**
- Backgrounds: **Haikei**, **SVGBackgrounds**, **Hero Patterns**
- Illustrations: **unDraw**
- Token glyphs: **SVGRepo** / **cryptoicons** (check licenses)

---

10) Roadmap → "Sexy & Smart" Dashboard

- Add focus drawer with full Lightweight-Chart per pair (OHLC overlay if available)
- Add aggregation tabs: by DEX, by Pair, All
- Arb Radar: highlight top cross-DEX spreads with 5-sec auto-cycle
- Keyboard nav: `j/k` for feed, `/` to search pair
- Metrics: FPS gauge, WS RTT, Kafka lag (in footer)
- Persist filters in URL (query-string)

---

11) Notes for Contributors & AIs

- Keep WS payloads small; batch where possible
- Favor SVG and CSS over heavy images; avoid global shadows
- All numbers use tabular numerals to prevent jitter
- Strict TypeScript interfaces for all frames (shared `types.ts`)
- Document topic contracts; version JSON if evolving

End of v1.